

スタートガイド (インストール、初期化、送受信)

インストール

1. Asset Store から `.unitypackage` をインポートします。
 - 古いバージョンからアップデートした場合は、以下のディレクトリを確認してください：
`Assets/MIDI/Plugins/Android` 古い、または重複している `.aar` ファイルがある場合は削除してください。
2. ターゲットプラットフォーム (iOS / Android / Standalone / WebGL / UWP) に切り替えます。
3. サンプルシーンをビルドして実行します：
 - `Assets/MIDI/Samples/Scenes/`

以前にオプションの依存関係 (Nearby など) をインストールしていた場合は、それらを最新の状態に更新してください。

MIDI 1.0 クイックスタート (受信 + オプションの BLE スキャン)

推奨されるライフサイクル：

```

using UnityEngine;
using jp.kshoji.unity.midi;

public sealed class Midi1QuickStart : MonoBehaviour, IMidiDeviceEventHandler, IMidiNoteOnHandler
{
    private void Awake()
    {
        // イベントハンドラオブジェクトを登録
        MidiManager.Instance.RegisterEventHandlerObject(gameObject);

        // MIDI の初期化
        MidiManager.Instance.InitializeMidi(() =>
        {
            #if (UNITY_ANDROID || UNITY_IOS || UNITY_WEBGL) && !UNITY_EDITOR
                // BLE MIDI のみ (サポートされている場合): 初期化完了後にスキャンを開始。
                MidiManager.Instance.StartScanBluetoothMidiDevices(0);
            #endif
        });
    }

    private void OnDestroy()
    {
        // MIDI の終了
        MidiManager.Instance.TerminateMidi();
    }

    public void OnMidiInputDeviceAttached(string deviceId)
    => Debug.Log($"入力デバイス接続: {deviceId}");

    public void OnMidiInputDeviceDetached(string deviceId)
    => Debug.Log($"入力デバイス切断: {deviceId}");

    public void OnMidiOutputDeviceAttached(string deviceId)
    => Debug.Log($"出力デバイス接続: {deviceId}");

    public void OnMidiOutputDeviceDetached(string deviceId)
    => Debug.Log($"出力デバイス切断: {deviceId}");

    public void OnMidiNoteOn(string deviceId, int group, int channel, int note, int velocity)
    => Debug.Log($"NoteOn デバイス:{deviceId} ch:{channel} note:{note} vel:{velocity}");
}

```

MIDI 1.0 送信

`MidiManager` を直接呼び出す方法:

```
// Note On を送信
MidiManager.Instance.SendMidiNoteOn(
    "deviceId",
    0 /*group*/,
    0 /*channel*/,
    60 /*note*/,
    127 /*velocity*/
);
```

Fluent API (`MidiSend`) を使う方法:

```
using jp.kshoji.unity.midi.util;

MidiSend.To("deviceId").Channel(0).NoteOn(60, 127);

// 最初の出力デバイスへ送信
MidiSend.ToFirstOutput().Channel(0).ControlChange(1, 64);
```

詳細は [ユーティリティ \(MidiNoteUtility / MidiMessageBuilder\)](#) を参照してください。

出力デバイスの ID は以下から取得できます：

- `MidiManager.Instance.OutputDeviceIdSet` (type: `HashSet<string>`)

エディタでのデバッグ

Play モード中、MIDI 入出力メッセージをリアルタイム確認するには **MIDI モニター** を使用します。

```
Window > MIDI > Monitor
```

詳細は [エディタツール \(MIDI モニター\)](#) を参照してください。

任意: Unity-MCP (AI / Cursor)

プロジェクトに [Unity-MCP](#) (`com.ivanmurzak.unity.mcp`) を入れ、**Window** → **AI Game Developer** を設定したうえで Cursor から:

1. Play Mode → `midi-init` → `midi-devices-list` → `midi-send-note` (C4) → `midi-monitor-read`

ビルド済み Player: `MidiMcpRuntimeBootstrap` を置き Host を LAN 上の MCP Server に設定し、コンポーネント配線は Editor で済ませてから制御ツールのみ使います。ネットワーク (`FEATURE_MIDI_NETWORK`): Editor `midi-net-hub-client-setup` → Player `midi-net-rtt`。MIDI 2.0: `midi2-devices-list` / `midi2-send-ump`。詳細は [MCP README](#)、[サンプルプロンプト](#)、[define 行列](#)。VST ホストは [Unity-VST3-Bridge](#) (デスクトップ Player ; Android 実機 MCP 対象外)。

MIDI 2.0 クイックスタート

```
using UnityEngine;
using jp.kshoji.unity.midi;

public sealed class Midi2QuickStart : MonoBehaviour, IMidi2DeviceEventHandler, IMidi2
{
    private void Awake()
    {
        MidiManager.Instance.RegisterEventHandleObject(gameObject);
        MidiManager.Instance.InitializeMidi2(() => { });
    }

    private void OnDestroy()
    {
        MidiManager.Instance.TerminateMidi2();
    }

    public void OnMidi2InputDeviceAttached(string deviceId)
        => Debug.Log($"MIDI2 入力接続: {deviceId}");

    public void OnMidi2InputDeviceDetached(string deviceId)
        => Debug.Log($"MIDI2 入力切断: {deviceId}");

    public void OnMidi2OutputDeviceAttached(string deviceId)
        => Debug.Log($"MIDI2 出力接続: {deviceId}");

    public void OnMidi2OutputDeviceDetached(string deviceId)
        => Debug.Log($"MIDI2 出力切断: {deviceId}");

    public void OnMidi2NoteOn(string deviceId, int group, int channel, int note, int
        => Debug.Log($"MIDI2 NoteOn デバイス:{deviceId} g:{group} ch:{channel} note:{r
    }
}
```

次に読むべきドキュメント

- [ゲームプレイ向けコンポーネント \(InputMap / NoteTracker / Filter\)](#)
- [SMF ツール \(SmfPlayer / MidiRecorder / TempoMapExtractor\)](#)
- [ユーティリティ \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [エディタツール \(MIDI モニター\)](#)
- [Unity-MCP 統合・サンプルプロンプト・define 行列](#)
- [プラットフォームと制限事項](#)

- ビルド後の処理 (PostProcessing) とスクリプト定義シンボル
- トランスポートとプラットフォームに関する注意点
- サンプル